

BLOC: ACS Improvement + PBS Integration

1 Asynchronous Common Subset in HoneyBadger

We first describe the asynchronous common subset (ACS) specification presented in HoneyBadgerBFT. Let $P_1, \dots, P_N$ be the parties, and assume at most $f < N/3$ are Byzantine. Communication is over authenticated asynchronous channels. Each correct party P_i provides an input value v_i to an ACS instance.

An ACS protocol satisfies the following properties:

1. **Validity:** If a correct node outputs a set V , then $|V| \geq N - f$, and V contains the inputs of at least $N - 2f$ correct nodes.
2. **Agreement:** If a correct node outputs V , then every correct node outputs V .
3. **Totality:** If $N - f$ correct nodes receive an input, then all correct nodes eventually produce an output.

HoneyBadger instantiates ACS from N parallel reliable broadcast instances and N parallel binary agreement instances. For each index $j \in \{1, \dots, N\}$, party P_j acts as the sender of an instance $RBC[j]$ with input v_j . In parallel, every party participates in a binary agreement instance $BA[j]$ that decides whether the value proposed by P_j is included in the final common subset.

1.1 Reliable Broadcast

For each sender index j , the instance $RBC[j]$ disseminates v_j so that if any correct node delivers a value from $RBC[j]$, then every correct node eventually delivers the same value.

HoneyBadger uses an erasure-coded reliable broadcast. Let P_j be the sender of $RBC[j]$.

1. **Encoding.** The sender applies an $(N - 2f, N)$ erasure code to v_j , producing shards

$$\{s_k\}_{k=1}^N,$$

so that any $N - 2f$ shards suffice to reconstruct v_j . In practice this can be instantiated with Reed-Solomon coding.

2. **Authentication.** The sender computes a Merkle tree over the shards and lets h_j be the Merkle root.
3. **Initial send.** For each party P_k , the sender transmits

$$\text{VAL}(h_j, b_k, s_k),$$

where b_k is the Merkle branch authenticating shard s_k under root h_j .

Each receiver then follows the standard VAL/ECHO/READY pattern:

1. Upon receiving a valid VAL from the designated sender, a node multicasts

$$\text{ECHO}(h_j, b_i, s_i).$$

2. Upon receiving a valid $\text{ECHO}(h_j, \cdot, \cdot)$ from $N - f$ distinct parties, a node:
 - (a) interpolates from any $N - 2f$ shards;
 - (b) recomputes the Merkle root and checks that it equals h_j ;
 - (c) if it has not already done so, multicasts

$$\text{READY}(h_j).$$

3. Upon receiving $f + 1$ matching $\text{READY}(h_j)$ messages, a node multicasts $\text{READY}(h_j)$ if it has not already sent it.
4. Upon receiving $2f + 1$ matching $\text{READY}(h_j)$ messages, a node waits until it has collected $N - 2f$ valid ECHO shards for root h_j , reconstructs v_j , and delivers the output of $RBC[j]$.

Let

$$\text{values}[j] \in \{v_j, \perp\}$$

denote the local state after reliable broadcast, where $\text{values}[j] = v_j$ means that the output of $RBC[j]$ has been delivered.

The use of erasure coding improves efficiency by having the sender distribute authenticated shards instead of transmitting the full proposal to every recipient.

1.2 Binary Agreement Array

In parallel with the N reliable-broadcast instances, the parties run

$$BA[1], \dots, BA[N].$$

For each index j , a node inputs

$$1 \text{ into } BA[j]$$

as soon as it delivers the output of $RBC[j]$.

This extra binary-agreement layer is necessary because RBC delivery is only a *local* event: one correct node may already have delivered $RBC[j]$ while another has not. If each node were to decide inclusion directly from its local delivery status, different correct nodes could reach different views of which proposals should be included. The role of $BA[j]$ is therefore to convert these different local observations into one common yes/no decision for proposer index j .

Nodes do *not* immediately input 0 for missing proposals. Instead, once a node has observed at least $N - f$ binary agreement instances decide 1, it inputs

$$0$$

to every still-uninitialized $BA[j]$ instance. This delayed-0 rule is the key step that prevents correct nodes from prematurely excluding slow but correct broadcasts.

Let

$$b_j \in \{0, 1\}$$

denote the decision of $BA[j]$, and define the agreed proposer set

$$S = \{j \mid b_j = 1\}.$$

1.3 One Concrete Binary Agreement Instantiation

HoneyBadger uses a randomized asynchronous binary agreement protocol based on a common coin. A concise message-level description is as follows.

Each instance $BA[j]$ proceeds in rounds $r = 0, 1, 2, \dots$. Let est_r be the current estimate.

1. On input $b_{in} \in \{0, 1\}$, set

$$est_0 := b_{in}.$$

2. In round r , multicast

$$\mathbf{BVAL}_r(est_r).$$

3. Upon receiving $\mathbf{BVAL}_r(b)$ from $f + 1$ distinct parties, multicast $\mathbf{BVAL}_r(b)$ if it has not yet been sent.

4. Upon receiving $\mathbf{BVAL}_r(b)$ from $2f + 1$ distinct parties, add b to

$$\mathbf{bin_values}_r.$$

5. Once $\mathbf{bin_values}_r \neq \emptyset$, multicast

$$\mathbf{AUX}_r(w)$$

for some $w \in \mathbf{bin_values}_r$.

6. Wait until at least $N - f$ valid $\mathbf{AUX}$ messages have been received and the values they carry form a set

$$\mathbf{vals} \subseteq \mathbf{bin_values}_r.$$

7. Invoke the round- r common coin and obtain

$$s_r \in \{0, 1\}.$$

8. If $\mathbf{vals} = \{b\}$, then set

$$est_{r+1} := b.$$

If additionally $b = s_r$, decide b .

9. Otherwise, set

$$est_{r+1} := s_r$$

and continue to the next round.

A node may temporarily observe both values as possible, i.e.,

$$\mathbf{bin_values}_r = \{0, 1\},$$

because different subsets of the network may have supplied enough support to propagate both $\mathbf{BVAL}_r(0)$ and $\mathbf{BVAL}_r(1)$. This does not break safety: it only means that, from that node's viewpoint, both outcomes are still consistent with the asynchronous message schedule. The common coin is what breaks this tie. When $\mathbf{vals} = \{0, 1\}$, all correct nodes use the same coin value s_r to choose the next estimate, which gradually aligns their states and ensures eventual termination.

At the level needed here, the important point is that each $BA[j]$ guarantees agreement, termination, and validity, and turns the local delivery status of $RBC[j]$ into a common yes/no inclusion decision.

1.4 ACS Output

After all $BA[j]$ instances terminate, each node waits for the outputs of $RBC[j]$ for all $j \in S$ and returns the common subset

$$\{v_j \mid j \in S\}.$$

This final waiting step is safe because if $BA[j]$ outputs 1, then at least one correct node must previously have had evidence that $RBC[j]$ should be included, and RBC guarantees that once any correct node delivers a value, all correct nodes eventually deliver the same value. Hence every node will eventually obtain the missing $RBC[j]$ outputs for all $j \in S$.

Thus ACS is realized by the composition

$$RBC[1..N] \longrightarrow BA[1..N] \longrightarrow \text{common subset output}.$$

2 Adapting ACS to BLOC

When applying this specification to BLOC, the parties are the operators of the DVT proposer cluster, and party P_i proposes an inclusion list IL_i for the current slot.

Each IL_i is constructed from the operator's local mempool view and contains encrypted placeholder transactions. The encrypted transactions are already protected at the transaction level by BLOC's batched threshold-encryption mechanism.

2.1 Reliable Broadcast of Inclusion Lists

For each proposer index j , operator P_j starts an instance

$$RBC[j]$$

with input

$$IL_j.$$

The same erasure-coded reliable broadcast is used:

1. P_j erasure-encodes IL_j into N authenticated shards.
2. P_j sends one shard and its Merkle proof to each operator via VAL.
3. Receivers relay valid shards using ECHO.
4. After receiving $N - f$ valid ECHO messages, a node reconstructs and validates the Merkle root and sends READY.
5. After receiving $f + 1$ matching READY messages, a node relays READY if it has not already done so.
6. After receiving $2f + 1$ matching READY messages and enough shards to decode, the node delivers IL_j .

Let

$$IL[j] \in \{IL_j, \perp\}$$

denote whether the reliable broadcast output for proposer j has been delivered locally.

2.2 Binary Agreement on Inclusion-List Membership

For each proposer index j , every operator runs one binary agreement instance $BA[j]$.

The input rule is:

$$\text{input}_j = \begin{cases} 1 & \text{if } IL[j] \neq \perp, \\ 0 & \text{otherwise.} \end{cases}$$

As before, the protocol does not eagerly input 0 for all missing broadcasts. Instead, once a node has observed at least $N - f$ binary agreement instances decide 1, it inputs 0 into every remaining uninitialized $BA[j]$ instance.

Let

$$b_j \in \{0, 1\}$$

be the output of $BA[j]$, and define

$$S = \{j \mid b_j = 1\}.$$

The ACS output for BLOC is then

$$\mathcal{I} = \{IL_j \mid j \in S\}.$$

Here ACS fixes the common set of candidate inclusion lists, while the later deterministic merge step only canonicalizes this already-agreed input into one ciphertext block body. Thus the agreement work is done by RBC and ABA; the merge rule merely ensures that all correct nodes serialize the agreed set in the same way.

3 PBS Integration in BLOC

We now describe how BLOC integrates with proposer-builder separation (PBS), given it previously produced a finalized plaintext transaction bundle for the current slot.

More precisely, consensus first fixes the ordered bundle of encrypted placeholder transactions, after which the selected batch is threshold-decrypted and each placeholder is deterministically resolved into its plaintext counterpart. The output of this process is therefore a finalized ordered plaintext transaction bundle that is ready to be shared with builders.

Let

$$P^s = (tx_1, \dots, tx_m)$$

denote the finalized plaintext bundle for slot s , where the order of the transactions has already been fixed by the proposer cluster and each element of P^s is the result of deterministic post-decryption materialization.

3.1 Protected Plaintext Prefix

The bundle P^s is treated as a mandatory prefix of the block for slot s . Any valid builder payload must therefore have the form

$$B^s = P^s \parallel \Sigma^s,$$

where Σ^s is an arbitrary suffix chosen by the builder.

Thus, the top of the block is already fixed before the builder enters the pipeline. PBS is used only to allow builders to compete on the remaining block space.

3.2 Builder Request

The proposer cluster forwards to builders:

1. the finalized plaintext prefix P^s ,
2. the prefix length m , and
3. a validity rule requiring that the candidate payload preserve P^s exactly in positions $1, \dots, m$.

Builders are therefore asked to construct bids only for payloads that satisfy this prefix constraint. In particular, they may not:

1. remove transactions from P^s ,
2. insert transactions before P^s , or
3. reorder transactions inside P^s .

They may, however, freely optimize the suffix Σ^s .

3.3 Bid Selection and implications

Each builder returns a bid for a payload satisfying the prefix rule. The proposer cluster validates that the candidate payload is of the form

$$B^s = P^s \parallel \Sigma^s.$$

Among all valid bids, the proposer cluster selects the one with highest value. Hence PBS remains active exactly where it is most useful: builders still compete to maximize the value of the suffix, even though the top of the block has already been fixed by BLOC. This enables backrunning MEV which boosts block profitability whilst keeping user transactions ordered by the protocol safe from frontrunning and sandwich attacks.

3.4 Using Bolt's Constraints API for Prefix Enforcement

A natural way to enforce the prefix constraint inside the PBS pipeline is to reuse Bolt's constraints API as a transport and verification layer, without adopting its preconfirmation or restaking architecture.

After the proposer cluster has produced the finalized plaintext prefix

$$P^s = (tx_1, \dots, tx_m),$$

it can encode this prefix as a single signed constraints bundle for slot s , with the `top` flag enabled. Concretely, the constraint message contains:

1. the proposer or validator public key,
2. the target slot s ,

3. the boolean `top` flag, and
4. the ordered list of transactions in P^s .

Bolt interprets `top=true` as a top-of-block constraint, which matches the BLOC requirement that P^s must occupy the first m positions of the block.

If desired, the proposer may authorize a separate sidecar key to submit these constraints on its behalf using Bolt’s delegation mechanism. This allows the validator signing path to remain separate from the component that interacts with builders and relays.

3.5 Concrete Proposer–Builder Flow

The proposer–builder interaction then proceeds in four steps:

1. The proposer cluster computes the finalized prefix P^s and submits it as a signed Bolt constraints bundle for slot s .
2. Relays expose these constraints to builders, either through a polling endpoint for slot-specific constraints or through Bolt’s constraints stream.
3. Each builder constructs a candidate payload of the form

$$B^s = P^s \parallel \Sigma^s,$$

optimizing only the suffix Σ^s .

4. The builder returns a bid together with inclusion proofs showing that the transactions of P^s appear in the required positions.

Thus, Bolt turns the ordinary PBS auction into a constrained auction: the prefix is fixed by the proposer cluster, and the builder competes only on the remaining block space.

3.6 Relay Interaction

On the relay side, Bolt exposes endpoints that let builders:

1. fetch active delegations for the slot,
2. fetch the constraints for a given slot, or
3. subscribe to a stream of newly submitted constraints.

This is the point where the BLOC prefix enters the PBS pipeline: once relayed, builders learn both which key is authorized to speak for the proposer and the exact ordered prefix they must preserve.

3.7 Proof Verification

Bolt extends the usual bid flow by allowing a builder bid to be accompanied by an `InclusionProofs` object. These proofs consist of transaction hashes, their generalized indexes, and Merkle hashes forming an SSZ Merkle multiproof against the block's `transactions_root`.

For BLOC, the proposer cluster verifies these proofs before bid selection. The validation rule is:

A bid is admissible only if its attached proofs certify that the transactions of P^s appear in positions $1, \dots, m$ of the candidate block body, in exactly the same order as in P^s .

Any bid whose proofs fail to establish this property is rejected before block selection. Bolt also mirrors this proof-carrying design on the relay block-submission path, so builders submit candidate blocks together with the corresponding inclusion proofs.